

Logging for ML Systems

Intuition

Logging is the process of tracking and recording key events that occur within an application for purposes such as monitoring, debugging, and analysis. It is far more powerful than simple print statements, as it allows developers to route specific information to designated outputs, apply structured and customizable formatting, and maintain consistent interfaces across the system. By providing visibility into an application's internal behavior, logging becomes an essential tool for diagnosing issues, understanding system performance, and extracting meaningful insights from runtime processes.

Components

There are a few overarching concepts to be aware of:

- **Logger:** emits the log messages from our application.
- **Handler:** sends log records to a specific location.
- **Formatter:** formats and styles of the log records.

There is so much [more](#) to logging such as filters, exception logging, etc. but these basics will allow us to do everything we need for our application.

Levels

Before creating a specialized and fully configured logger, let's first examine what logged messages look like using the basic configuration.

```
import logging
import sys

# Create super basic logger
logging.basicConfig(stream=sys.stdout, level=logging.DEBUG)

# Logging levels (from lowest to highest priority)
logging.debug("Used for debugging your code.")
logging.info("Informative messages from your code.")
logging.warning("Everything works but there is something to be aware of.")
logging.error("There's been a mistake with the process.")
logging.critical("There is something terribly wrong and process may
terminate.")
```

```
DEBUG:root:Used for debugging your code.
```

INFO:root:Informative messages from your code.

WARNING:root:Everything works but there is something to be aware of.

ERROR:root:There's been a mistake with the process.

CRITICAL:root:There is something terribly wrong and process may terminate.

These are the basic logging levels, where **DEBUG** is the lowest priority and **CRITICAL** is the highest. We configured our logger using *basicConfig* to emit log messages to standard output (i.e., the terminal console), but we could also direct them to other streams or even to a file.

We also set the logging level to **DEBUG**, which means all log messages are displayed, since DEBUG is the lowest level. If we had set the level to **ERROR**, then only messages with severity **ERROR** and **CRITICAL** would be shown.

```
import logging
import sys

# Create super basic logger
logging.basicConfig(stream=sys.stdout, level=logging.ERROR)

# Logging levels (from lowest to highest priority)
logging.debug("Used for debugging your code.")
logging.info("Informative messages from your code.")
logging.warning("Everything works but there is something to be aware of.")
logging.error("There's been a mistake with the process.")
logging.critical("There is something terribly wrong and process may
terminate.")
```

ERROR:root:There's been a mistake with the process.

CRITICAL:root:There is something terribly wrong and process may terminate.

Configuration

First, we'll set the location of our logs in our config.py script:

```
# root/config.py
LOGS_DIR = Path(BASE_DIR, "logs")
LOGS_DIR.mkdir(parents=True, exist_ok=True)
```

Next, we'll configure the logger for our application:

```
# root/config.py
import logging
import sys
logging_config = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "minimal": {"format": "%(message)s"},
        "detailed": {
            "format": "%(levelname)s %(asctime)s
[%(name)s:%(filename)s:%(funcName)s:%(lineno)d]\n%(message)s\n"
        },
    },
    "handlers": {
        "console": {
            "class": "logging.StreamHandler",
            "stream": sys.stdout,
            "formatter": "minimal",
            "level": logging.DEBUG,
        },
        "info": {
            "class": "logging.handlers.RotatingFileHandler",
            "filename": Path(LOGS_DIR, "info.log"),
            "maxBytes": 10485760, # 1 MB
            "backupCount": 10,
            "formatter": "detailed",
            "level": logging.INFO,
        },
        "error": {
            "class": "logging.handlers.RotatingFileHandler",
            "filename": Path(LOGS_DIR, "error.log"),
            "maxBytes": 10485760, # 1 MB
            "backupCount": 10,
            "formatter": "detailed",
            "level": logging.ERROR,
        },
    },
    "root": {
        "handlers": ["console", "info", "error"],
        "level": logging.INFO,
        "propagate": True,
    },
}
```

1. [Lines 6–11]: define two different **Formatters** (determine format and style of log messages), minimal and detailed, which use various **LogRecord attributes** to create a formatting template for log messages.
2. [Lines 12–35]: define the different **Handlers** (details about location of where to send log messages):
 - i. **console**: sends log messages (using the **minimal** formatter) to the **stdout** stream for messages above level **DEBUG** (ie. all logged messages).
 - ii. **info**: send log messages (using the **detailed** formatter) to **logs/info.log** (a file that can be up to **1 MB** and we'll backup the last **10** versions of it) for messages above level **INFO**.
 - iii. **error**: send log messages (using the **detailed** formatter) to **logs/error.log** (a file that can be up to **1 MB** and we'll backup the last **10** versions of it) for messages above level **ERROR**.
3. [Lines 36–40]: attach our different handlers to our root **Logger**.

We chose to use a dictionary to configure our logger but there are other ways such as Python script, configuration file, etc.

We can load our logger configuration dict like so:

```
import logging
# Logger
logging_config = {...}
logging.config.dictConfig(logging_config)
logger = logging.getLogger()
# Sample messages (note that we use configured `logger` now)
logger.debug("Used for debugging your code.")
logger.info("Informative messages from your code.")
logger.warning("Everything works but there is something to be aware of.")
logger.error("There's been a mistake with the process.")
logger.critical("There is something terribly wrong and process may
terminate.")
```

DEBUG	Used for debugging your code.	config.py:71
INFO	Informative messages from your code.	config.py:72
WARNING	Everything works but there is something to be aware of.	config.py:73
ERROR	There's been a mistake with the process.	config.py:74
CRITICAL	There is something terribly wrong and process may terminate.	config.py:75

Our logged messages become stored inside the respective files in our logs directory:

```
logs/  
├── info.log  
└── error.log
```

And since we defined a detailed formatter, we would see informative log messages like these:

```
INFO 2020-10-21 11:18:42,102 [config.py:module:72]
```

Informative messages from your code.

Implementation

In our project, we can replace all of our print statements into logging statements:

```
print("☑ Training complete!")
```

———— becomes: ————

```
from config import logger  
logger.info("☑ Training complete!")
```

All our log messages are at the INFO level but while developing we may have had to use DEBUG levels and we also add some ERROR or CRITICAL log messages if our system behaves in an unintended manner.

- **what:** log all the necessary details you want to surface from our application that will be useful *during* development and *afterwards* for retrospective inspection.
- **where:** the best practice is to not clutter our modular functions with log statements. Instead, we should log messages outside of small functions and inside larger workflows. For example, there are no log messages inside any of our scripts except the main.py and train.py files. This is because these scripts use the smaller functions defined in the other scripts (data.py, evaluate.py, etc.). If we ever feel that we need to log within our other functions, then it usually indicates that the function needs to be broken down further.

When it comes to saving our logs, we could simply [upload](#) our logs to a cloud blog storage (ex. S3 or Google Cloud Storage). Or for a more production-grade logging option, we could consider the [Elastic stack](#).